LLMs for 6G Networks: Fine-Grained Telecom Traffic Generation and Forecasting

Final Project Report

Ahcene LOUBAR & Maryam BACHTTI

Laboratoire des Signaux et Systèmes (L2S), CentraleSupélec

Supervisor: Alexis Aravanis

Project objective. Starting from a coarse-grained operator dataset, the goal of this project is to build a realistic traffic forecasting pipeline for telecom networks. The report addresses three linked questions: (i) why point forecasting is insufficient for network operation and why probabilistic forecasting is needed, especially for upper-quantile or worst-case traffic estimation; (ii) how coarse 5-minute averaged traffic can be augmented into finer-grained stochastic traffic using an adapted Interrupted Poisson Process (IPP) framework inspired by the KTH reference paper; and (iii) how classical statistical models, neural sequence models, and large language model based time-series forecasters compare on the resulting telecom prediction task.

March 2026

Contents

1 Introduction **3**

2 Problem Formulation and Motivation **3**
2.1 Why point forecasting is not sufficient . . . 3
2.2 Probabilistic forecasting and pinball loss . . . 4
2.3 Project question . . . 4

3 Dataset Description and Exploratory Analysis **5**
3.1 Available data . . . 5
3.2 Preprocessing steps . . . 5
3.3 Macroscopic aggregation . . . 5

4 Why Coarse Measurements Need Augmentation **6**

5 KTH Reference Framework: Interrupted Poisson Process Traffic Generation **7**
5.1 ON/OFF activity model . . . 7
5.2 Number of arrivals in a slot . . . 7
5.3 Traffic amount per arrival . . . 8
5.4 Why the original KTH setup cannot be used unchanged . . . 8

6 Engineering Adaptation of the KTH Framework to the Bouygues Dataset **8**
6.1 Overview of the final implemented generator . . . 8
6.2 Variance proxy from a single time series . . . 9
6.3 Second-moment matching used in the final code . . . 9
6.4 Event-driven simulation with session spreading . . . 10
6.5 Exact mean conservation . . . 10
6.6 Why aggregation solved the “too many zeros” issue . . . 10

7 Implementation Architecture **11**
7.1 Code organization . . . 11
7.2 Core synthesis pipeline . . . 12
7.3 Selected implementation excerpts . . . 12

8 Forecasting Dataset and Experimental Protocol **14**
8.1 Final fine-grained dataset . . . 14
8.2 Forecasting horizons . . . 14
8.3 Train-test protocol . . . 14

9 Forecasting Models **15**
9.1 SARIMA baseline . . . 15
9.2 PyTorch LSTM . . . 15
9.3 Chronos V1 and Chronos 2 . . . 15

10 Evaluation Metrics **15**
10.1 RMSE 16
10.2 Pinball loss 16

11 Quantitative Results **16**
11.1 Multi-step forecasting results (12-step horizon) 16
11.2 Single-step forecasting results (1-step horizon) 18

12 Discussion **19**
12.1 What the results say about the augmentation stage 19
12.2 Why did SARIMA remain so competitive? 20
12.3 Why did the LSTM underperform? 20
12.4 Why Chronos 2 is particularly relevant 20
12.5 Probabilistic forecasting: what was done and what remains 21

13 Limitations **21**

14 Conclusion **21**

15 Hyperparameters used in the final implementation **22**

16 Compact pipeline summary for oral presentation **23**

1 Introduction

Future mobile networks are expected to combine higher traffic demand, tighter quality-of-service constraints, and stricter energy-efficiency requirements. The rough dataset description provided with the Bouygues material explicitly frames the problem as *predictive dimensioning*: the operator does not want to dimension blindly or react only after congestion appears, but rather anticipate future demand in order to provision radio resources, plan upgrades, and rationalize deployment decisions. In the accompanying project slides, this is linked to the uneven daily distribution of traffic, the rapid growth of smartphone consumption, and the operational need to dimension the network close to its true demand rather than oversizing everything by default.

In that context, forecasting is not merely a statistical exercise. It directly affects how safely a network can activate energy-saving mechanisms, including sleep modes for network elements. The KTH reference paper used in this project states the problem clearly: base stations consume a large fraction of network energy, and activating sleep modes can save energy, but incorrect activation under bursty traffic may create delays for users. The paper therefore motivates traffic-aware, risk-aware control policies and emphasizes that the available operator traffic measurements are often too coarse to describe the short-time behavior relevant for sleep mode decisions.

This project starts from the same tension. The raw Bouygues traffic traces are coarse-grained averages, whereas the operational question is inherently finer in time and more risk-sensitive. Predicting only the mean traffic is not enough when the downstream action depends on whether the network can safely absorb short traffic peaks. What matters is not only the central forecast but also the distribution of future traffic, and especially the upper part of that distribution. In other words, the relevant question is not "what is the average traffic likely to be?" but rather "what is a plausible worst-case traffic level that should still be safely supported?"

The project therefore has two tightly coupled parts:

1) **Traffic augmentation:** adapt the KTH Interrupted Poisson Process (IPP) framework to transform coarse telecom measurements into a finer-grained stochastic traffic signal that preserves the macro envelope while recovering realistic burstiness.

2) **Forecasting:** benchmark a classical statistical model (SARIMA), a trained recurrent neural network (LSTM), and pretrained foundation models for time series (Chronos V1 and Chronos 2) on the generated telecom sequences.

The final pipeline is therefore both methodological and application-driven. Methodologically, it shows how to bridge a granularity mismatch between industrial telecom data and learning-based forecasters. Operationally, it moves the study closer to the 6G use case where forecasting should support proactive and risk-aware resource management.

2 Problem Formulation and Motivation

2.1 Why point forecasting is not sufficient

In a telecom network, many management actions are asymmetric in cost. Underestimating future load can be much more damaging than overestimating it slightly. If a base station, radio chain, or

other capacity-enhancing element is put into sleep mode too aggressively, and a traffic burst arrives shortly afterward, the network may incur queueing delay, degraded service, or missed energy-delay trade-offs. This risk-aware perspective is explicit in the KTH paper, where incorrect sleep mode decisions are associated with delayed users and where a dedicated *risk of decision making* metric is proposed to quantify how dangerous a wrong action may be under a given traffic regime.

That perspective immediately suggests a forecasting requirement stronger than simple point estimation. A single mean forecast collapses the full uncertainty of the future into one number. It cannot distinguish between a stable low-load scenario and a volatile scenario whose mean is identical but whose peaks are much higher. For safe network operation, the upper quantiles of the future traffic distribution are often more informative than the mean:

- the mean describes the central tendency of expected load;
- upper quantiles describe conservative or worst-case operating conditions;
- the gap between them captures uncertainty and burstiness.

Formally, if Y_{t+h} denotes the future traffic at horizon h, a point forecast targets something like $\hat{y}_{t+h} \approx \mathbb{E}[Y_{t+h} \mid \mathcal{F}_t]$. In contrast, risk-aware forecasting seeks quantiles such as $Q_\alpha(Y_{t+h} \mid \mathcal{F}_t)$ for $\alpha > 0.5$, for example $\alpha = 0.9$ or 0.95, because these are directly linked to a conservative capacity or sleep-mode decision.

2.2 Probabilistic forecasting and pinball loss

This is precisely why probabilistic forecasting matters in this project. The Chronos paper formulates time-series forecasting as prediction of a *joint predictive distribution* over future values rather than only a single point estimate. The same distinction applies to neural quantile models more generally. A natural scoring rule for such forecasts is the *pinball loss*, defined for a quantile level $\alpha \in (0, 1)$ by

$$\mathcal{L}_\alpha(y, \hat{q}_\alpha) = \max\left(\alpha(y - \hat{q}_\alpha), (\alpha - 1)(y - \hat{q}_\alpha)\right), \tag{1}$$

where $\hat{q}_\alpha$ is the predicted α-quantile. Minimizing this loss encourages correct quantile calibration. The larger the chosen α, the more the forecast targets the upper tail.

The motivating narrative of this report is therefore the following:

> *To support safe activation of energy-saving mechanisms in future mobile networks, forecasting should not be limited to the mean. It should quantify uncertainty and upper-quantile behavior. However, the available operator data is too coarse for this directly, which motivates a traffic synthesis stage before forecasting.*

2.3 Project question

The project can be summarized by the following question:

> Starting from a real operator dataset observed at coarse granularity, can we build a fine-grained stochastic telecom traffic dataset that preserves the macro behavior of the original traces, and can that dataset be used to benchmark classical, neural, and foundation-model forecasters in a way that is relevant to risk-aware 6G network operation?

This report answers that question through a complete pipeline: preprocessing, aggregation-aware traffic synthesis, forecasting, and evaluation.

3 Dataset Description and Exploratory Analysis

3.1 Available data

The Bouygues material provided for the project includes:

- a spreadsheet describing the antenna layout and sector/site information,
- a traffic history file (`histo_trafic.csv`),
- a rough explanatory presentation on predictive dimensioning.

According to the project statement, the traffic dataset contains 86 antenna sectors distributed over 29 sites, with most sectors having 287 measurements. After preprocessing and cleaning, the exploration notebook reported 24,486 valid records over 86 unique sectors. The key variable used in the experiments is `trafic_mbps`, interpreted as a coarse traffic measurement in Mbps.

A central instruction from the supervisor was to *treat the Bouygues traffic values as if they were 5-minute averages*, and then generate an instantaneous or finer-grained signal "on top" of these averages, in the spirit of Section II-C and Figure 6(a) of the KTH reference paper.

3.2 Preprocessing steps

The raw file required several cleaning steps before any modeling could be performed:

1) removal of unnamed columns,
2) cleaning of French date strings,
3) translation of month names into English to allow robust `datetime` parsing,
4) chronological sorting by sector and timestamp,
5) conversion of `trafic_mbps` into numeric format,
6) removal of missing values.

These operations produce sector-wise coarse time series

$$X_s(k), \qquad s \in \{1, \ldots, S\},\ S = 86, \tag{2}$$

where each observation is a coarse traffic measurement for sector s at time index k.

3.3 Macroscopic aggregation

A key turning point in the project was the realization that the diagnostic comparison requested by the teacher should be read at the *aggregate* network scale rather than only at the level of one isolated sector. This is consistent with the teacher's example, where the red simulated signal

oscillates around a global traffic envelope instead of displaying the sparse and highly intermittent behavior of a single sector.

We therefore define the aggregated macro-series

$$X_{\text{agg}}(k) = \sum_{s=1}^{S} X_s(k). \tag{3}$$

This aggregation is not a cosmetic change: it fundamentally changes the stochastic regime. If sectors are only weakly dependent, means add linearly while relative variability decreases. In practice, aggregation makes the global traffic smoother and more visually realistic, which proved essential to reproduce the type of signal expected by the supervisor.

Figure 1 shows the aggregated Bouygues traffic over time. The figure exhibits three important characteristics: a clear long-term growth trend, non-negligible fluctuations around that trend, and local dips that show the series is not simply monotonic. This is exactly the kind of macro-load profile for which fine-scale stochastic augmentation becomes meaningful.

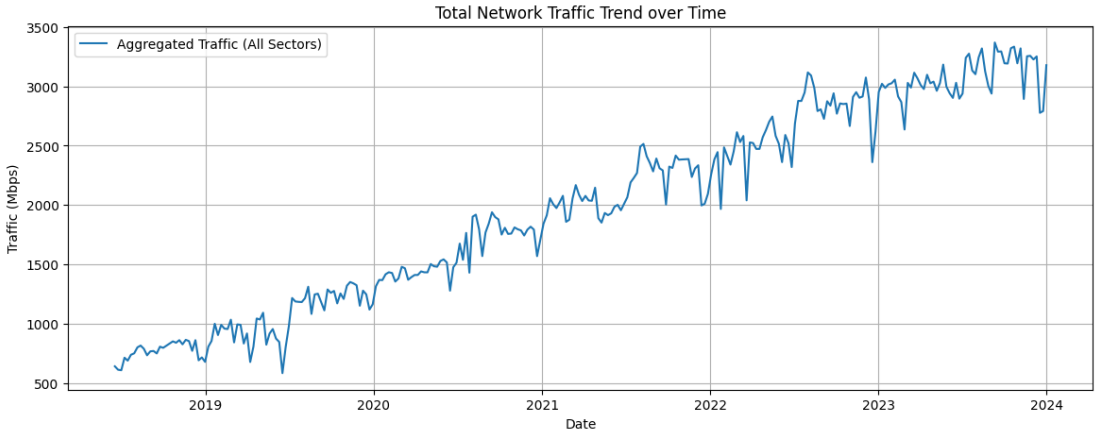


Figure 1: Aggregated traffic obtained by summing all sectors at each timestamp. The macro-series shows a clear growth trend together with short-term fluctuations.

4 Why Coarse Measurements Need Augmentation

The KTH paper makes a point that is directly applicable to this project: coarse operator measurements provide the long-term and daily structure of traffic, but they are too coarse to describe the short-time behavior relevant for millisecond- to second-scale decisions. In the Stockholm dataset used in the paper, samples were collected every 5 minutes, which was already too coarse for sleep-mode management. The exact same difficulty appears in the Bouygues dataset once the forecasting problem is interpreted in a 6G operational setting.

Suppose the only information available for a time slot of duration $T = 300$ seconds is its average rate y_t in Mbps. This average carries no direct information about:

- how many arrivals occurred inside the slot,
- whether those arrivals were clustered or spread out,
- whether the traffic was smooth or bursty,

- whether a sleep mode action would have been safe in the first seconds of the next interval.

From the perspective of a forecasting model, this creates two difficulties:

1) **Learning difficulty:** models trained only on coarse averages cannot learn realistic intra-slot burstiness.

2) **Evaluation difficulty:** forecasting only the average hides the variability that matters for worst-case decisions.

The purpose of the synthesis stage is therefore not to fabricate arbitrary high-frequency noise. It is to build a fine-grained process that is consistent with the coarse measurements while injecting a physically meaningful burst structure.

5 KTH Reference Framework: Interrupted Poisson Process Traffic Generation

5.1 ON/OFF activity model

Following Section II-C of the KTH reference paper, traffic activity is modeled by a two-state continuous-time Markov chain (CTMC):

$$\text{OFF} \xrightarrow{\tau} \text{ON}, \qquad \text{ON} \xrightarrow{\zeta} \text{OFF}. \tag{4}$$

The stationary probabilities of the two states are

$$P_{\text{ON}} = \frac{\tau}{\tau + \zeta}, \qquad P_{\text{OFF}} = \frac{\zeta}{\tau + \zeta}. \tag{5}$$

When the process is in the ON state, arrivals occur according to a Poisson process with rate λ; when it is in the OFF state, no arrival occurs. This is the classical Interrupted Poisson Process (IPP).

5.2 Number of arrivals in a slot

If U denotes the number of arrivals during a slot of duration T, the KTH paper gives the mean

$$\mathbb{E}[U] = \lambda \frac{\tau}{\tau + \zeta} T = \lambda P_{\text{ON}} T, \tag{6}$$

and, under the large-slot approximation $(\tau + \zeta)T \gg 1$, the variance

$$\text{Var}(U) \approx \lambda \frac{\tau T}{\tau + \zeta} \left[1 + \frac{2\lambda\zeta}{(\tau + \zeta)^2} \right]. \tag{7}$$

This model is attractive because it is both bursty and analytically tractable: the OFF state introduces silence periods, while the ON state produces clusters of arrivals.

5.3 Traffic amount per arrival

Each arrival carries a mark or requested amount ψ_j. In the original KTH derivation, the marks are assumed exponentially distributed. If Ψ is the total traffic amount over a slot,

$$\Psi = \sum_{j=1}^{U} \psi_j, \tag{8}$$

then the slot-level moments satisfy

$$\mathbb{E}[\Psi] = \mathbb{E}[U]\mathbb{E}[\psi], \tag{9}$$

$$\mathrm{Var}(\Psi) = \mathbb{E}[U]\mathrm{Var}(\psi) + \mathrm{Var}(U)\mathbb{E}[\psi]^2. \tag{10}$$

The paper then proposes to match these moments to the mean and variance extracted from the operator dataset at each time index. If multi-day statistics are available, this yields closed-form expressions for λ and $\mathbb{E}[\psi]$ once τ and ζ are fixed.

5.4 Why the original KTH setup cannot be used unchanged

The Bouygues dataset does not have the same statistical structure as the dataset used in the paper. The KTH framework assumes that for each time-of-day index i, one can collect a set of measurements across many days,

$$O_i = \{o_1, o_2, \ldots, o_\kappa\}, \tag{11}$$

and then compute both $\mathbb{E}(O_i)$ and $\mathrm{Var}(O_i)$. In the Bouygues case, we only have a single time series per sector. Therefore, there is no natural collection of repeated observations for each time-of-day index from which a per-slot variance can be estimated in the exact same way.

This is the main reason an engineering adaptation was needed: the KTH mathematics was the right starting point, but the available dataset required a stabilized and practical variant of the original recipe.

6 Engineering Adaptation of the KTH Framework to the Bouygues Dataset

6.1 Overview of the final implemented generator

The final implementation is best understood as a *guided adaptation* of the KTH framework rather than a literal copy. The implementation keeps the core physical assumptions:

- ON/OFF CTMC activity,
- Poisson arrivals in the ON state,
- exponential marks for arrival amounts,
- event-driven simulation inside each coarse slot.

However, several engineering modifications were required to make the model stable and compatible with the Bouygues dataset:

1) a variance proxy is built from the available single time series,

2) a feasibility floor is added so that the KTH moment equations remain solvable,

3) arrival rates are stabilized by lower and upper clamps,

4) arrivals are spread over a short session duration instead of appearing as one-bin spikes,

5) each synthesized slot is rescaled so that its average matches the observed coarse value exactly.

These modifications are not arbitrary hacks: each one addresses a concrete pathology encountered during the implementation.

6.2 Variance proxy from a single time series

Since the Bouygues data does not provide repeated measurements for each time-of-day index, the variance term required by the KTH moment-matching equations must be approximated. The implemented solution is a rolling variance proxy. If x_t denotes the coarse mean signal, the empirical variance proxy is computed over a moving window of length k and then regularized by two floors:

a) a generic floor proportional to the local mean,

b) a KTH feasibility floor derived from the CTMC correction term.

In code terms, the resulting proxy is the maximum of the rolling variance and those floors. This makes the second-moment matching numerically safe.

6.3 Second-moment matching used in the final code

The final implementation does not stop at mean matching. Given a target coarse rate r and a variance proxy v, and fixing τ, ζ, and T, it uses the KTH-inspired decomposition

$$r = P_{\mathrm{ON}} \lambda \psi, \tag{12}$$

$$v = \frac{2P_{\mathrm{ON}} \lambda \psi^2}{T} + \frac{2\zeta}{\tau T(\tau + \zeta)} r^2. \tag{13}$$

The second term plays the role of a CTMC correction due to the ON/OFF structure. Defining

$$V_{\mathrm{diff}} = v - \frac{2\zeta}{\tau T(\tau + \zeta)} r^2, \tag{14}$$

the implementation solves

$$\psi = \frac{V_{\mathrm{diff}} T}{2r}, \qquad \lambda = \frac{r}{P_{\mathrm{ON}} \psi}. \tag{15}$$

The code then stabilizes λ by imposing a minimum value and a maximum cap. Whenever λ is clamped, ψ is adjusted so that the target mean is still preserved.

6.4 Event-driven simulation with session spreading

A naive implementation that places all the traffic of one arrival into a single fine-grained bin produces unrealistic impulses and too many near-zero bins. To correct this, the final generator uses an event-driven simulation and spreads each arrival over a short constant session duration. Let D denote that duration. If an arrival with amount ψ_j starts at time t_j, its rate contribution over the interval $[t_j, t_j + D]$ is approximated as flat:

$$\text{rate}_j \approx \frac{\psi_j}{D}. \tag{16}$$

This avoids pathological spikes and yields a more realistic fine-scale signal. Sessions truncated by the slot boundary simply contribute their nominal rate to the remaining bins of the slot.

6.5 Exact mean conservation

Even after moment matching and event-driven simulation, stochastic sampling can make the realized average of a generated slot differ slightly from the observed coarse target. To eliminate that mismatch, the final code applies an exact mean correction. If a slot produces fine-scale samples $x_{t,1}, \ldots, x_{t,m}$ with realized mean

$$\bar{x}_t = \frac{1}{m} \sum_{k=1}^{m} x_{t,k}, \tag{17}$$

then the slot is rescaled as

$$x_{t,k} \leftarrow x_{t,k} \cdot \frac{y_t}{\bar{x}_t}, \tag{18}$$

where y_t is the observed coarse traffic for that slot. This guarantees that re-aggregating the synthesized signal recovers the original coarse series exactly up to floating-point precision.

6.6 Why aggregation solved the "too many zeros" issue

One of the most important empirical findings of the project was that sector-level simulation often looked too sparse compared with the teacher's example, with many low bins and strong peaks. This is not necessarily a coding bug; it is also a scale issue. A single sector can indeed have intermittent behavior. By contrast, the sum of many sectors behaves like a macro-flow with many overlapping contributions. The law of large numbers reduces relative burstiness and fills the apparent "holes" of the signal.

This is why the aggregated simulation finally resembled the supervisor's target figure much more closely. Figure 2 shows one of the intermediate diagnostics used to tune the ON/OFF parameters. Figure 3 shows the final aggregated comparison at 60-second granularity.

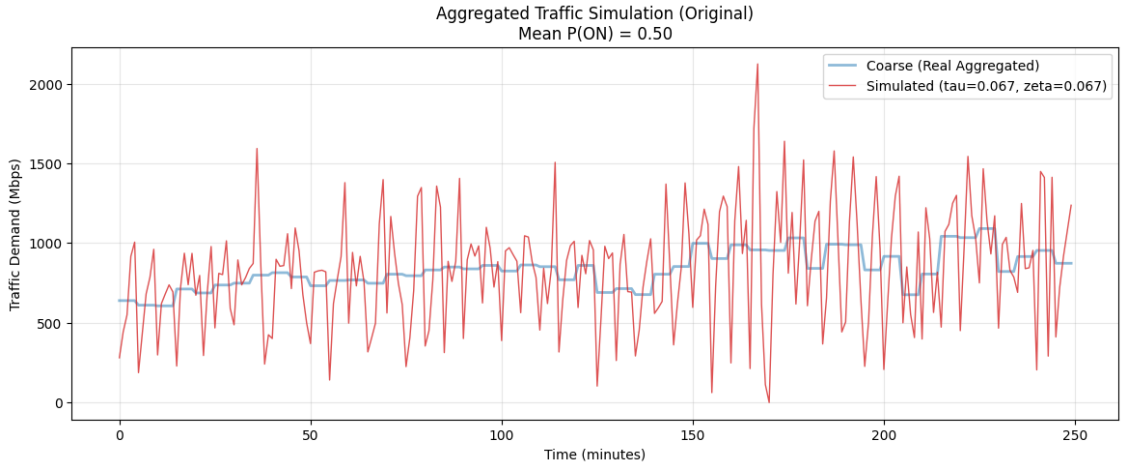


Figure 2: Diagnostic aggregated simulation used during parameter tuning. The chosen regime corresponds to $\tau = 0.067$ and $\zeta = 0.067$, yielding $P_{\mathrm{ON}} \approx 0.5$.

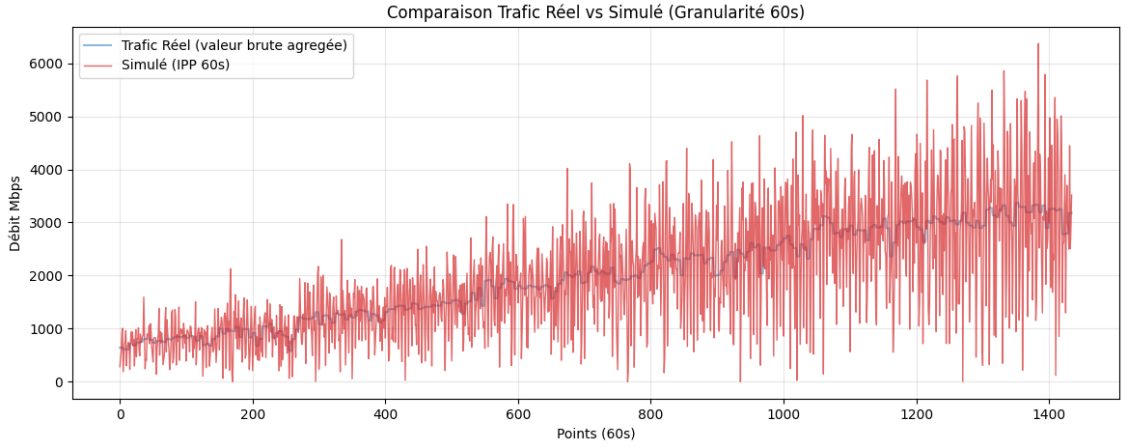


Figure 3: Comparison between aggregated coarse traffic and synthesized 60-second IPP traffic. The generated signal follows the macro envelope while restoring fast stochastic fluctuations.

7 Implementation Architecture

7.1 Code organization

The implementation was organized into lightweight Python modules, each responsible for one stage of the pipeline. Table 1 summarizes the role of the main files.

Table 1: Main Python modules used in the project.

File	Role in the pipeline
`config.py`	Stores the KTH/IPP configuration parameters: coarse slot length T, fine resolution dt, CTMC rates τ and ζ, fixed minimum arrival rate, variance scaling factor, and session duration.
`utils.py`	Provides helper functions, especially the rolling variance proxy, fine-to-coarse consistency checks, and summary error metrics.
`kth_ipp.py`	Core traffic generator: moment matching, event-driven ON/OFF simulation, arrival generation, session spreading, and exact mean conservation.
`data_loader.py`	Handles Bouygues traffic loading, aggregation, synthesis of fine-grained data, scaling, and preparation of train/validation/test splits.
`evaluate.py`	Computes MAE, RMSE, MAPE, and the implemented pinball loss at quantile level 0.5.
Forecast notebooks	Explore the data, validate the synthesized signal, train the LSTM, and evaluate SARIMA, Chronos V1, and Chronos 2.

7.2 Core synthesis pipeline

The final augmentation pipeline can be summarized as follows:

1) load and clean the Bouygues sector-level dataset;

2) optionally aggregate sectors to inspect macro behavior;

3) for each sector (or for the macro-series during diagnostics), compute a rolling variance proxy;

4) solve the KTH-inspired moment equations for each slot;

5) simulate an event-driven ON/OFF arrival process inside the slot;

6) spread arrivals over a short session duration;

7) rescale the slot to enforce exact coarse consistency;

8) concatenate all slots into a fine-grained series;

9) save the resulting augmented dataset for forecasting.

7.3 Selected implementation excerpts

The report does not reproduce the full codebase line by line, but the three excerpts below capture the essential logic.

Excerpt 1: loading and aggregating the Bouygues data

```
def load_and_aggregate_data(filepath="data/histo_trafic.csv"):
    df = pd.read_csv(filepath, sep=";", encoding="latin1")
    df = df.loc[:, ~df.columns.str.contains("^Unnamed")]
    df["tstamp_clean"] = df["tstamp"].str.replace(
        r"^[a-zA-Z ]+\s+", "", regex=True
    )
    # French month name conversion omitted here for brevity
    df["tstamp"] = pd.to_datetime(df["tstamp_clean"], format="%d %B %Y"
        )
    df = df.drop(columns="tstamp_clean")
    df = df.sort_values(["secteur", "tstamp"])
    df["trafic_mbps"] = pd.to_numeric(df["trafic_mbps"], errors="coerce
        ")
    df = df.dropna(subset=["trafic_mbps"])
    df_agg = df.groupby("tstamp")["trafic_mbps"].sum().reset_index()
    return df_agg.sort_values("tstamp")
```

Excerpt 2: variance proxy and feasibility stabilization

```
def rolling_variance_proxy(coarse_mean, k, var_floor_ratio, *, tau,
    zeta, T, eps=1e-6):
    x = np.asarray(coarse_mean, dtype=float)
    s = pd.Series(x)
    v = s.rolling(window=k, min_periods=max(3, k // 4)).var(ddof=0).
        to_numpy()
    global_v = float(np.nanvar(x)) if np.isfinite(np.nanvar(x)) else
        0.0
    v = np.where(np.isfinite(v), v, global_v)

    floor_generic = (var_floor_ratio * np.maximum(x, 0.0)) ** 2
    v = np.maximum(v, floor_generic)

    correction = (2.0 * zeta) / (tau * T * (tau + zeta))
    floor_kth = (correction + eps) * (np.maximum(x, 0.0) ** 2)
    v = np.maximum(v, floor_kth)
    return v
```

Excerpt 3: exact mean conservation in the generator

```
slot_rate = simulate_ipp_slot(sp, p, rng)
target = coarse_mbps[t]
m = float(slot_rate.mean())

if target <= 0:
    slot_rate[:] = 0.0
```

```
else:
    if m <= 0:
        slot_rate[:] = target
    else:
        slot_rate *= (target / m)
```

These excerpts highlight the actual engineering philosophy of the project: preserve the realism of the KTH stochastic model, but stabilize it with simple and transparent numerical safeguards.

8 Forecasting Dataset and Experimental Protocol

8.1 Final fine-grained dataset

Once the augmentation pipeline had been validated visually and numerically, the synthesized telecom traces were saved into a fine-grained dataset (`augmented_data.csv`) used by the forecasting experiments. This dataset contains fine-resolution traffic time series for the 86 sectors.

The role of the aggregate notebook was primarily diagnostic: it demonstrated that the adapted IPP generator behaves correctly at macro scale and visually reproduces the type of signal requested by the supervisor. The forecasting experiments, however, were run sector by sector on the saved fine-grained dataset.

8.2 Forecasting horizons

Two complementary prediction tasks were evaluated:

- **Multi-step forecasting:** predict the last 12 timestamps of each sector using the 60 previous timestamps as context.
- **Single-step forecasting:** predict strictly the final timestamp of each sector.

These two settings answer slightly different questions. The 12-step task evaluates a model's ability to track a future trajectory over a short horizon, while the 1-step task isolates the final prediction error requested in the early stage of the project.

8.3 Train-test protocol

For each sector, the forecasting notebook extracted the corresponding fine-grained time series, sorted it chronologically, and reserved the final $60 + 12$ points as the evaluation block. The preceding part of each series was used to build sliding windows for training the LSTM. In the notebook, windows were extracted with a stride of 20 points in order to increase training diversity while keeping the process computationally light.

The input and output lengths were fixed to:

$$\text{context_length} = 60, \qquad \text{pred_length} = 12. \tag{19}$$

Each sector was scaled independently to the range $[-1, 1]$ before being fed to the LSTM, and inverse scaling was applied to the model outputs before metric computation.

9 Forecasting Models

9.1 SARIMA baseline

The first benchmark is SARIMA, used as a strong classical baseline. This is an important part of the report because it anchors the evaluation in a model family that does not rely on deep representation learning. In time-series forecasting, strong classical baselines are essential; without them, improvements claimed by neural or foundation models can be misleading.

SARIMA is local, interpretable, and often surprisingly competitive on relatively short univariate series. In this project, it provides the main statistical reference against which the neural models were judged.

9.2 PyTorch LSTM

The second model is a trainable recurrent neural network implemented in PyTorch. The architecture is a two-layer LSTM with hidden size 64 and output size equal to the 12-step prediction horizon. The model was trained for 30 epochs using the Adam optimizer with learning rate 0.002 and mean squared error loss.

The LSTM is the only model in the benchmark that is explicitly trained on the synthesized telecom dataset. Its role is important for two reasons:

1) it tests whether the synthesized dataset is useful for supervised learning,

2) it provides a learned benchmark that is more specialized than SARIMA but less general than Chronos.

9.3 Chronos V1 and Chronos 2

The final two models are Amazon Chronos V1 (`amazon/chronos-t5-small`) and Chronos 2. Chronos is a pretrained probabilistic forecasting framework that tokenizes scaled time-series values into discrete bins and applies a transformer language model over those tokens. During inference, multiple future trajectories can be sampled and transformed back to numeric forecasts. The original Chronos paper emphasizes that this language-modeling approach produces probabilistic forecasts and is particularly attractive in zero-shot scenarios where no task-specific training is performed.

This makes Chronos especially relevant for the project narrative: unlike the LSTM, which had to be trained on the synthesized telecom dataset, Chronos can be evaluated directly in a pretrained zero-shot regime. This lets us test whether a time-series foundation model transfers effectively to augmented telecom traffic.

10 Evaluation Metrics

10.1 RMSE

The root mean squared error (RMSE) was used to measure point-forecast accuracy:

$$\mathrm{RMSE} = \sqrt{\frac{1}{H}\sum_{h=1}^{H}(y_{t+h} - \hat{y}_{t+h})^2}, \tag{20}$$

where H is the forecast horizon. RMSE penalizes large errors more heavily than mean absolute error and therefore highlights failures on large traffic deviations.

10.2 Pinball loss

To keep a probabilistic score in the comparison, the implementation also computed the pinball loss. In the current evaluation code, the pinball score was reported at quantile level 0.5, which corresponds to the median. For a general quantile level α, the loss is

$$\mathrm{PBL}_\alpha(y, \hat{q}_\alpha) = \max\{\alpha(y - \hat{q}_\alpha), (\alpha - 1)(y - \hat{q}_\alpha)\}. \tag{21}$$

This deserves a precise interpretation. The overall *motivation* of the project is clearly upper-quantile forecasting for worst-case traffic scenarios. However, the specific evaluation code implemented during the benchmark stage reports pinball loss at $\alpha = 0.5$, which is a median-oriented score. This still makes sense as a first probabilistic benchmark because it keeps the comparison consistent across models. Nevertheless, the natural next step is to extend the exact same framework to higher quantiles such as 0.9 or 0.95, which would align even more directly with risk-aware sleep-mode decisions.

11 Quantitative Results

11.1 Multi-step forecasting results (12-step horizon)

Table 2 reports the average RMSE and pinball loss across the 86 sectors for the 12-step forecasting task.

Table 2: Average performance over 86 sectors for the 12-step forecasting task. The reported pinball loss is computed at quantile level 0.5. Lower is better.

Model	Avg. RMSE (12-step)	Avg. Pinball Loss (12-step)
SARIMA	15.5699	6.3047
LSTM	17.8863	7.4964
Chronos V1	16.1626	6.5225
Chronos 2	15.7372	6.3858

Several conclusions emerge from this table.

First, **SARIMA is a very strong baseline**. It obtains the best average RMSE and the best average pinball loss on the 12-step task, albeit by a small margin. This is an important

empirical result because it shows that on relatively short univariate telecom sequences, a classical local model remains highly competitive.

Second, **Chronos 2 is very close to SARIMA**. Its average RMSE is only slightly larger, and its pinball loss is also close. This is encouraging because Chronos 2 is used in a zero-shot regime: it did not require task-specific training on the telecom dataset.

Third, **Chronos V1 also performs well** but is marginally behind Chronos 2, which is consistent with the intuition that the newer Chronos generation is better calibrated for transfer.

Finally, **the LSTM underperforms the other models** on this task. This does not mean that recurrent models are unsuitable for telecom forecasting in general, but rather that in this setup the available amount of task-specific training data and the chosen architecture were not enough to outperform the best baseline and the pretrained foundation models.

Figure 4 gives qualitative examples of the 12-step forecasts for three representative sectors.

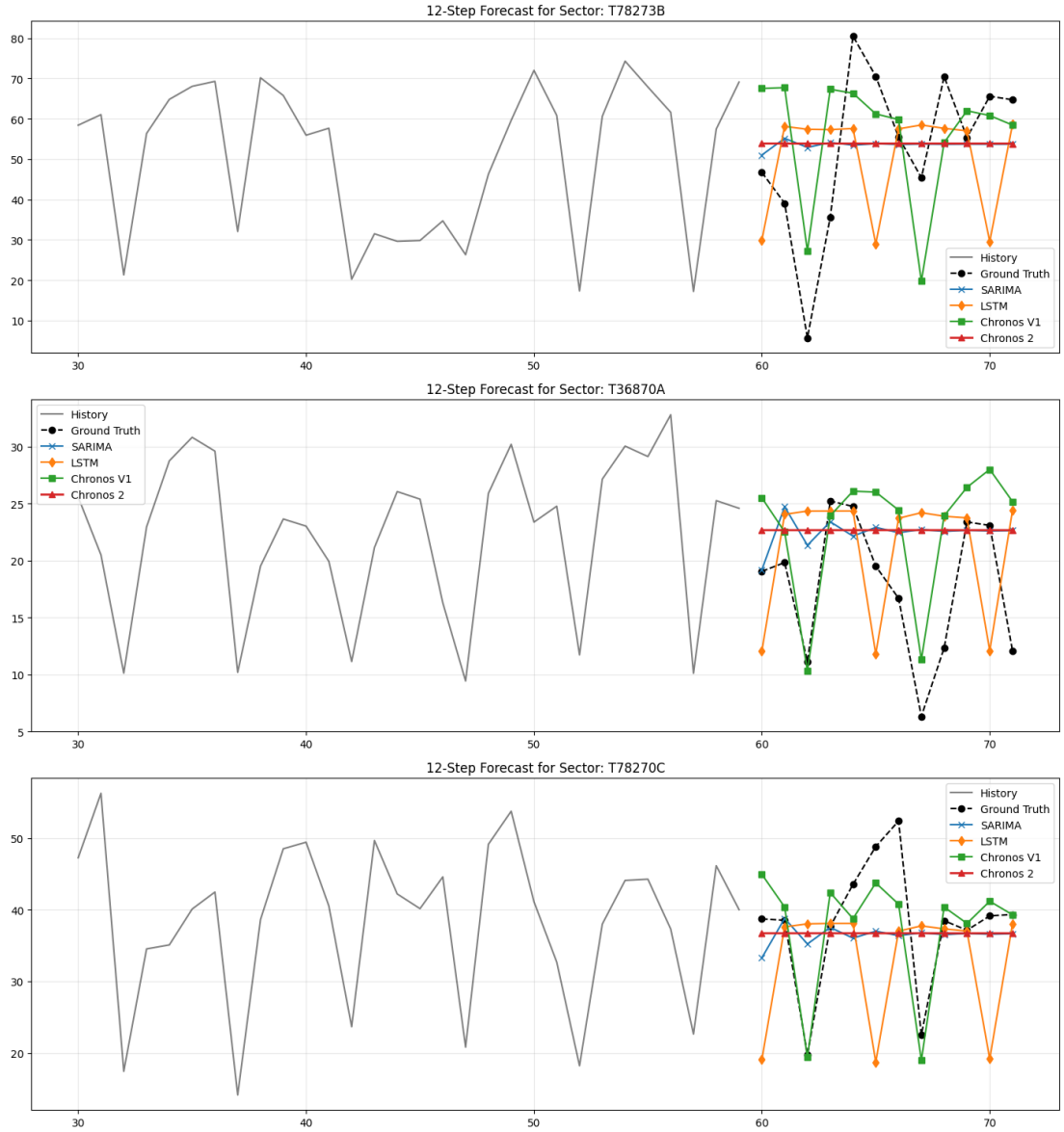


Figure 4: Examples of 12-step forecasts on three sectors. The figure compares the ground truth with SARIMA, LSTM, Chronos V1, and Chronos 2.

The figure illustrates an important practical point: the models differ not only in overall error but also in *forecast shape*. SARIMA often tracks local continuation well. The LSTM tends to produce flatter or less adaptive trajectories in difficult cases. Chronos V1 and Chronos 2 generally remain close to the recent history while still responding to sector-specific amplitude changes.

11.2 Single-step forecasting results (1-step horizon)

Table 3 reports the average performance over the 86 sectors for the strictly final-timestamp prediction task.

Table 3: Average performance over 86 sectors for the 1-step forecasting task. The reported pinball loss is computed at quantile level 0.5. Lower is better.

Model	Avg. RMSE (1-step)	Avg. Pinball Loss (1-step)
SARIMA	12.9776	6.4888
LSTM	27.7163	13.8582
Chronos V1	14.9924	7.4962
Chronos 2	11.5108	5.7554

The single-step task produces an even clearer ranking:

- **Chronos 2 is the best overall model**, both in RMSE and pinball loss.
- **SARIMA remains strong** and is second best overall.
- **Chronos V1 is competitive but behind Chronos 2**.
- **LSTM is clearly the weakest model in this setup**.

Figure 5 shows one-step examples on the same three representative sectors.

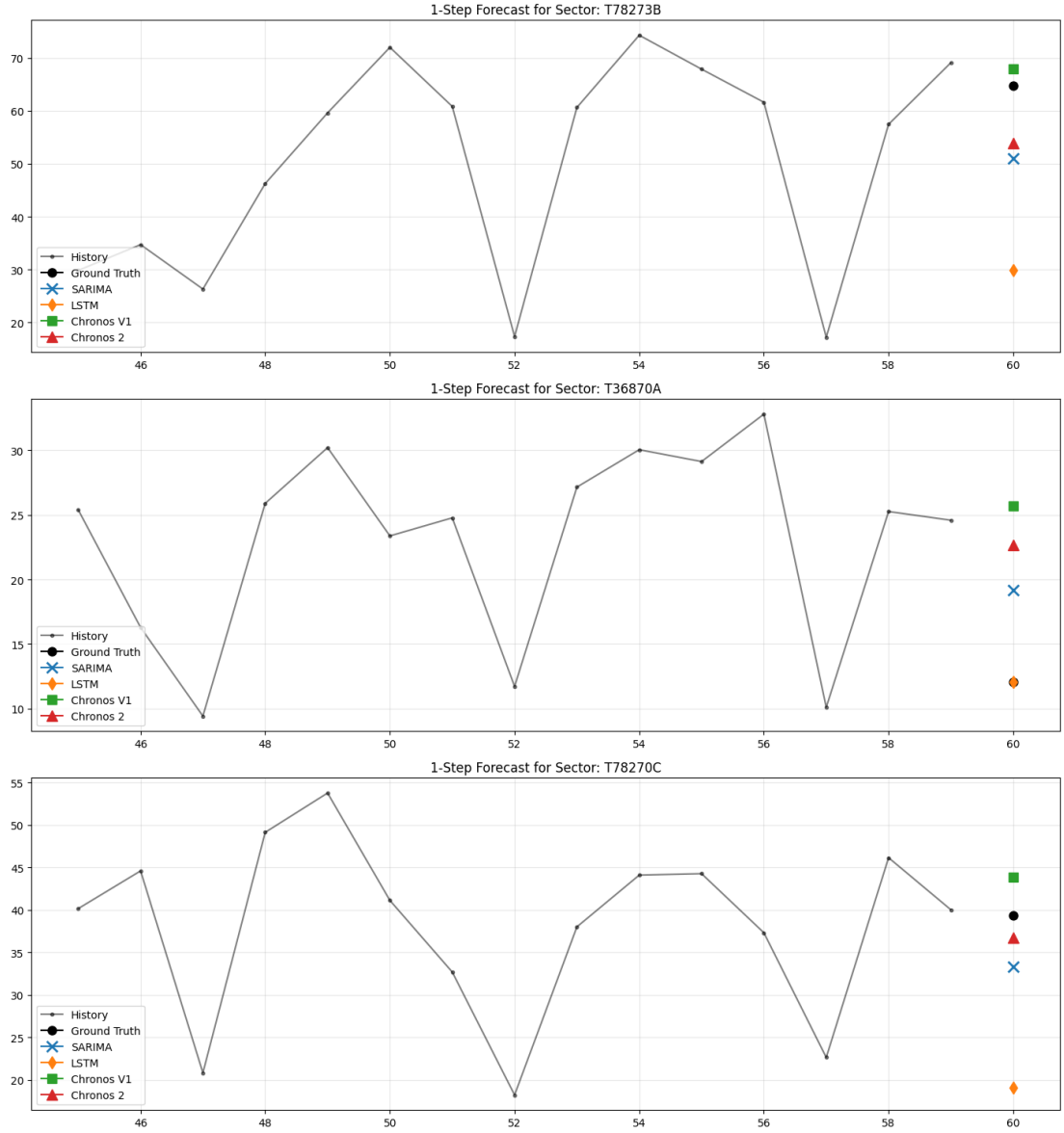


Figure 5: Examples of 1-step forecasts on three sectors. Chronos 2 is consistently close to the target final value and outperforms the other models on average.

This result is arguably the cleanest headline of the project. For the exact "last-measurement" task originally requested by the supervisor, Chronos 2 achieved the best average RMSE and the best average pinball loss among the tested models.

12 Discussion

12.1 What the results say about the augmentation stage

The forecasting benchmark should not be interpreted independently of the augmentation stage. A large part of the project was devoted to constructing a fine-grained dataset that is physically plausible and numerically stable. Without that stage, the LSTM and Chronos models would have been trained or tested on a signal that does not correspond to the true operational objective.

The synthesis stage therefore plays two roles simultaneously:

1) it turns the available data into a form usable for models that need finer temporal resolution,

2) it reframes the forecasting task toward risk-aware traffic variation rather than simple coarse averaging.

From this point of view, one of the most valuable outcomes of the project is not only the final score table, but the fact that the entire data pipeline now matches the intended telecom use case much more closely.

12.2 Why did SARIMA remain so competitive?

SARIMA performing strongly is not a failure of the project; on the contrary, it is scientifically useful. It shows that the augmented telecom traffic still contains strong local temporal structure that classical models can exploit. In short-horizon univariate forecasting, a strong statistical baseline is often difficult to beat.

This is precisely why including SARIMA was necessary. If only LSTM and Chronos had been evaluated, it would have been impossible to tell whether a sophisticated model was truly learning anything beyond a strong local baseline.

12.3 Why did the LSTM underperform?

Several factors likely contributed to the LSTM's weaker results:

- the effective training corpus, although larger than the raw data, remains small compared with the amount of data usually needed by global neural forecasters;
- the model was trained with mean squared error rather than directly on quantiles;
- the network architecture was intentionally lightweight;
- the telecom signal remains noisy and heterogeneous across sectors.

Thus, the LSTM should be interpreted as a meaningful supervised benchmark rather than as the definitive deep-learning solution for this problem.

12.4 Why Chronos 2 is particularly relevant

Chronos 2 is especially interesting because it performs strongly without task-specific retraining. This suggests that pretrained time-series foundation models can transfer useful generic forecasting priors to telecom signals, even when those signals were obtained through an augmentation pipeline rather than directly measured at fine resolution.

In practical terms, this is promising for 6G-style pipelines. If a foundation model can already provide strong forecasts on a new telecom dataset with little or no task-specific training, then the deployment pipeline becomes much simpler: less manual tuning, less retraining overhead, and faster benchmarking across sites or use cases.

12.5 Probabilistic forecasting: what was done and what remains

The supervisor explicitly insisted on the operational importance of upper-quantile forecasting for worst-case traffic scenarios. This report fully adopts that problem formulation. The entire motivation of the project is framed around the idea that predicting only the mean is insufficient for safe sleep-mode decisions.

At the same time, it is important to be precise about what was implemented in the current benchmark. The reported probabilistic metric is pinball loss at quantile level 0.5. This keeps the evaluation in the probabilistic-forecasting family, but it is still centered on the median. Therefore:

- **the conceptual target of the project** is upper-quantile forecasting for risk-aware control;
- **the current implementation** validates the pipeline using RMSE and median pinball loss;
- **the natural next step** is to report pinball loss at higher quantiles such as 0.9 and 0.95, especially for Chronos and for any future quantile-trained LSTM variant.

This is not a contradiction; it is a standard progression. First, validate that the augmentation and forecasting pipelines are sound. Then, move from median-oriented probabilistic evaluation to the upper-tail forecasts that are most relevant for deployment.

13 Limitations

No final report is complete without acknowledging its limits. The present work has several:

1) **Granularity is synthesized, not measured.** The fine-grained telecom traces are generated from a stochastic model, not directly observed from the operator.
2) **Variance is approximated.** The KTH framework was adapted using a rolling variance proxy because the Bouygues dataset does not provide repeated samples per time-of-day index.
3) **Sector interactions are simplified.** Forecasting was performed sector by sector, whereas real networks may show stronger spatial dependence.
4) **Pinball loss was reported at** 0.5**.** This is useful but not yet the final upper-quantile evaluation directly aligned with risk-aware deployment.
5) **The LSTM was not quantile-trained.** A dedicated quantile LSTM or distributional recurrent model would be a more direct probabilistic comparison.

Despite these limitations, the current pipeline already constitutes a coherent and operationally relevant benchmark.

14 Conclusion

This project addressed a real and nontrivial telecom forecasting problem: how to move from coarse operator measurements to a forecasting pipeline that is meaningful for 6G-style, risk-aware network control.

The work proceeded in three major stages.

First, the Bouygues traffic dataset was cleaned and explored. This made it clear that the available measurements were coarse averages and therefore insufficient, on their own, for the intended operational objective.

Second, an adapted version of the KTH Interrupted Poisson Process framework was implemented to synthesize fine-grained stochastic traffic from the coarse series. The final implementation combined KTH-inspired moment matching with a rolling variance proxy, event-driven ON/OFF simulation, session spreading, and exact mean conservation. A key empirical insight was that aggregated macro-level validation was essential to obtain a realistic visual match with the supervisor's reference figure.

Third, the resulting synthesized telecom dataset was used to compare SARIMA, a trained LSTM, Chronos V1, and Chronos 2. The benchmark showed that SARIMA remains a strong classical baseline, that the LSTM did not outperform the other models in the present setup, and that Chronos 2 achieved the best overall single-step performance while remaining highly competitive on the 12-step task.

The main value of the project lies in the complete pipeline it established: from coarse-grained operator traffic to fine-grained stochastic augmentation, then from fine-grained traffic to forecasting benchmarks that are directly connected to risk-aware 6G network operation. The next natural extension is to push the evaluation from the median toward upper quantiles, which would align even more directly with worst-case traffic management and safe sleep-mode control.

15 Hyperparameters used in the final implementation

Table 4 summarizes the main parameter choices used in the final implementation and experiments.

Table 4: Main parameters used in the implemented pipeline.

Component	**Value / setting**
Coarse slot duration T	300 s
Fine resolution for macro validation	60 s
CTMC rates for final macro validation	$\tau = 0.067$, $\zeta = 0.067$
Stationary ON probability	$P_{\mathrm{ON}} = 0.5$
Variance scaling factor	0.5
Minimum fixed arrival rate in configuration	0.5
Session mean duration	30 s
LSTM context length	60
LSTM prediction length	12
LSTM hidden size	64
LSTM number of layers	2
LSTM epochs	30
LSTM optimizer / learning rate	Adam / 0.002
Chronos V1 model	`amazon/chronos-t5-small`
Chronos 2 model	`amazon/chronos-2`

16 Compact pipeline summary for oral presentation

For presentation purposes, the entire project can be summarized in the following sequence:

1) Clean the Bouygues traffic history and build sector-level coarse time series.

2) Observe that the data is too coarse for risk-aware forecasting or sleep-mode reasoning.

3) Use the KTH IPP model as the stochastic physical prior for bursty arrivals.

4) Adapt KTH to the Bouygues dataset using a rolling variance proxy and exact mean preservation.

5) Validate the synthesized signal at aggregated macro scale and save the final fine-grained dataset.

6) Benchmark SARIMA, LSTM, Chronos V1, and Chronos 2 on 12-step and 1-step forecasting.

7) Evaluate with RMSE and pinball loss, and interpret the results from the perspective of future upper-quantile, risk-aware forecasting.

References

[1] M. Masoudi, E. Soroush, J. Zander, and C. Cavdar, *Digital Twin Assisted Risk-Aware Sleep Mode Management Using Deep Q-Networks*, arXiv:2208.14380, 2022.